F3

Faculty of Electrical Engineering
Department of Measurement

Bachelor's Thesis

LionKey (security key) NFC support

Matyáš Martan
Open Informatics - Internet of Things

May 2026

https://github.com/Matesin/bachelor-thesis

Supervisor: Ing. Jan Sobotka, PhD.

BACHELOR'S THESIS ASSIGNMENT

I. Personal and study details

Student's name: **Martan Matyáš** Personal ID number: **525903**

Faculty / Institute: **Faculty of Electrical Engineering**

Department / Institute: **Department of Measurement**

Study program: **Open Informatics**

Specialisation: **Internet of Things**

II. Bachelor's thesis details

Bachelor's thesis title in English:

LionKey (security key) NFC support

Bachelor's thesis title in Czech:

LionKey (bezpečnostní klíč) rozšíření NFC

Name and workplace of bachelor's thesis supervisor:

Ing. Jan Sobotka, Ph.D. Department of Measurement FEE

Name and workplace of second bachelor's thesis supervisor or consultant:

Date of bachelor's thesis assignment: **09.02.2026** Deadline for bachelor thesis submission: _____

Assignment valid until: **by the end of summer semester 2026/2027**

Head of department's signature

Vice-dean´s signature on behalf of the Dean

III. Assignment receipt

The student acknowledges that the bachelor's thesis is an individual work.
The student must produce his thesis without the assistance of others, with the exception of provided consultations.
Within the bachelor's thesis, the author must state the names of consultants and include a list of references.

Date of assignment receipt

Martan Matyáš

Student's signature

BACHELOR‘S THESIS ASSIGNMENT

I. Personal and study details

Student's name: **Martan Matyáš** Personal ID number: **525903**

Faculty / Institute: **Faculty of Electrical Engineering**

Department / Institute: **Department of Measurement**

Study program: **Open Informatics**

Specialisation: **Internet of Things**

II. Bachelor’s thesis details

Bachelor’s thesis title in English:

LionKey (security key) NFC support

Bachelor’s thesis title in Czech:

LionKey (bezpečnostní klíč) rozšíření NFC

Guidelines:

For the LionKey project: An open-source FIDO2 USB Security Key, implement basic support for NFC technology. The work consists of the following parts:
1) Familiarization with the existing codebase and the technologies needed to implement NFC.
2) Implement basic NFC functionality on the STEVAL-ST25R3916B hardware.
3) Demonstrate basic functionality - e.g., authentication on https://webauthn.io

Bibliography / sources:

https://github.com/pokusew/lionkey
https://dspace.cvut.cz/bitstream/handle/10467/123886/F3-DP-2025-Endler-Martin-FIDO2_USB_Security_Key.pdf?sequence=-1&isAllowed=

FAKULTA ELEKTROTECHNICKÁ
FACULTY OF ELECTRICAL ENGINEERING
Technická 2
166 27 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Martan Matyáš
Personal number:	525903
Programme name:	Open Informatics

hereby declare that the Bachelor's Thesis titled

LionKey (security key) NFC support

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 12.05.2026

Matyáš Martan
.....
student's signature

Errata

Version of this text: **2026-06-5 12:30 CEST**

The latest version of this text can be found at the following URL:
`https://github.com/Matesin/bachelor-thesis`

Acknowledgement

I would like to thank my supervisor, Ing. Jan Sobotka, Ph.D. for his guidance and assistance, doc. Ing. Jiří Novák, Ph.D. for introducing me to the topic of this thesis and to Ing. Martin Endler for his patience, enthusiasm and for the trust he put in me in continuing his LionKey project.

Furhermore, I would like to thank my family, especially my father, for supporting me at all times.

Abstrakt / Abstract

Tato bakalářská práce se zabývá implementací podpory technologie NFC pro hardwarový bezpečnostní klíč LionKey, který slouží jako externí FIDO2 autentizátor. Cílem práce je rozšířit existující zařízení o komunikační rozhraní umožňující jeho použití také se zařízeními vybavenými NFC čtečkou, zejména pak s mobilními telefony.

Práce popisuje teoretické principy technologie NFC, související komunikační standardy a jejich vztah k použití bezpečnostních klíčů v autentizačních mechanismech. Dále se věnuje návrhu a realizaci NFC rozhraní pro platformu LionKey, integraci do existující architektury klíče a ověření funkčnosti výsledného řešení.

Výsledkem práce je funkční implementace NFC komunikace pro klíč LionKey s využitím platformy STEVAL-D25R16B. Výsledné řešení je veřejně dostupné na GitHubu a rozšiřuje možnosti použití zařízení na stránkách podporujících WebAuthn.

Klíčová slova: FIDO2, WebAuthn, CTAP2, Near-Field Communication, NFC, bezpečnostní klíč, passkeys, autentizátor, bezheslová autentizace, vícefaktorová autentizace, MFA, ISO/IEC-14443, LionKey

Překlad titulu: LionKey (bezpečnostní klíč) rozšíření NFC

This bachelor's thesis focuses on implementing NFC support for the LionKey hardware security key, which serves as an external FIDO2 external authenticator. The aim of this thesis is to extend the capabilities of the existing device by implementing an NFC communication interface on top of the existing solution, allowing the key to communicate with devices equipped with an NFC reader, especially mobile phones.

The thesis describes the theoretical principles of NFC technology, the related communication standards, and their relevance to the use of security keys in authentication mechanisms. It also focuses on the design and implementation of an NFC interface for the LionKey platform, its integration into the existing device architecture, and the verification of the resulting solution.

The result of this thesis is a functional implementation of NFC communication for the LionKey security key using the STEVAL-D25R16B platform. The resulting implementation is publicly available on GitHub. It extends the device's capabilities by allowing its use with WebAuthn-allowed services.

Keywords: FIDO2, WebAuthn, CTAP2, Near-Field Communication, NFC, security key, passkeys, authenticator, passwordless authentication, multi-factor authentication, MFA, ISO/IEC-14443, LionKey

Contents /

Tables / Figures

Chapter 1
Introduction

Passkeys have become a **popular means of authentication** on the Internet. Along with more conventional means of passkeys, users are also adopting **hardware security keys** as a **safer way to store their credentials**. Hardware security keys may provide an additional layer of security compared to platform authenticators. They offer stronger isolation of private keys and ensure their non-exportability, thereby reducing the risk of credential compromise even if the host device is not trusted.

One such project is the **LionKey**, an open-source hardware security key developed at **CTU FEE** by **Ing. Martin Endler**. The device is compliant with FIDO2 standards; however, it currently **supports only CTAP over USB** (CTAPHID). This means that the key cannot be used with devices that do not support USB-based communication, notably mobile phones.

1.1 Thesis Objective

The objective of this thesis is to **extend** the **communication capabilities** of the LionKey hardware security key by **implementing an NFC-based communication** interface on top of the existing CTAPHID solution, in order to further improve user experience when using the authenticator.

The NFC interface is realized using the **STEVAL-D25R3916B** board, which is interfaced with the ST **Nucleo-H533RE** microcontroller in accordance with the original specifications of the LionKey project.

1.2 Thesis Structure

The thesis is organized into six main sections.
The **Introduction** outlines the context of the work and formulates the objectives of the thesis.
Chapter 2 presents the theoretical background of passkeys and motivates their use in modern authentication systems.
Chapter 3 introduces the fundamental principles of NFC technology, focusing on aspects relevant to the proposed solution.
Chapter 4 details the design, hardware used, and implementation of the proposed solution.
Chapter 5 describes the results achieved by this implementation and its applicability to practice.
Chapter 6 summarizes the results achieved and discusses the general outcome of the thesis.

Chapter 2
Passkeys

The Internet has become an integral part of daily life for most of the world's population. In December 2025, approximately **6.047 billion people** (73.2% of the global population) were **Internet users**, representing a 14% increase compared to 2020 [1]. This growth, along with an increasing reliance on online services, has been accompanied by an increase in cyberattacks and personal data breaches.

This development highlights a fundamental weakness of password-based authentication. According to the Verizon Data Breach Investigations Report, approximately **88% of breaches** in 2025 involved **stolen credentials** [2]. Passwords are therefore inherently vulnerable to theft, reuse, and brute-force attacks.

In response, organizations have **increasingly adopted multi-factor authentication (MFA)**, which supplements passwords with additional verification factors such as one-time codes, authentication applications, or trusted devices. The adoption of MFA has grown significantly; for example, **83% of organizations required MFA** for all access to IT in 2024 [3]. In the Czech Republic, the use of MFA by organizations is **required by law** since 2025. Despite variations across surveys, it is clearly observable that **MFA has become a widely used safety measure**.

2.1 The Advantages of Passkeys

Passkeys – standardized by the FIDO Alliance – have emerged as a response to the limitations of traditional multi-factor authentication (MFA). Many conventional MFA methods are considered to be insufficiently secure, while simultaneously introducing unnecessary complexity to the authentication process, thus worsening the user experience.

The FIDO Alliance **defines a passkey** as "...authentication **credential** based on FIDO standards that allows a user to **log into apps and websites** with the same process they use to unlock their device (biometrics, PIN or pattern). Passkeys rely on **asymmetric cryptography**, where the private key is securely stored on the user's authenticator, while the corresponding public key is registered with the service ". [1]

According to data published by the FIDO Alliance, passkey support has already been **adopted** by approximately a **fifth** of the world's **top 100 websites** and services [4]. Furthermore, major technology providers, including Apple, Google, and Microsoft, have integrated passkey support into their platforms, allowing further support for this authentication method.

[1] `https://fidoalliance.org/passkeys/`

2.1.1 Hardware security keys

Although passkeys are often implemented as platform authenticators integrated into user devices, the **FIDO2** framework also supports **external authenticators** in the form of **hardware security keys**. These devices **store cryptographic credentials** on dedicated hardware and **perform authentication independently** of the host system and only **upon user interaction**, providing an additional layer of security.

Currently, the hardware security key market is dominated by several major vendors, most notably **Yubico** with its Security Key Series. Other notable solutions include the Google Titan Security Key developed by Google and the SecurID authenticators made by RSA Security. In addition, several smaller or less widely cited manufacturers provide compatible FIDO2 security keys, including Feitian Technologies, Swissbit, and TrustKey.

Quantitative estimates of market share are difficult to obtain, as publicly available data are limited and often based on proprietary methodologies. **Estimates suggest**, as of April 2026 [5] that **Yubico accounts for approximately 32% and RSA Security for 24%** of the 2FA **solutions**. However, these figures reflect only the customer base observed by the source platform and should be interpreted as indicative rather than representative of the global market share of hardware security keys.

A distinguishing **characteristic** of most commercially available hardware security keys is their **closed-source firmware**. Open-source alternatives do exist but remain rare: the first publicly known example was SoloKeys (developed 2018–2024, now discontinued), followed by OpenSK by Google and Nitrokey. To the best of our knowledge, no open-source C-based implementation of CTAP2 over NFC for a hardware security key was publicly available at the time of writing.

2.2 The FIDO2 Framework

The **FIDO2** framework, developed by the FIDO Alliance in collaboration with the W3C, is the current **industry standard** for **strong, phishing-resistant and passwordless authentication**. It is an open framework that uses public-key cryptography.

FIDO2 is composed of two complementary core specifications:

- **WebAuthn (Web Authentication):** A W3C standard API that enables web applications to create and use public-key credentials. It defines how a browser or platform (the “Client”) communicates with a web service (the “Relying Party”) to register and authenticate users.
- **CTAP (Client-to-Authenticator Protocol):** A FIDO Alliance specification that defines how a Client communicates with an authenticator (such as a hardware security key or a smartphone) to fulfill WebAuthn requests.

  The CTAP standard defines two primary versions:

- **CTAP1 (formerly U2F):** Supports legacy FIDO U2F credentials.
- **CTAP2:** The current standard for FIDO2, which improves user verification methods, most notably by implementing a PIN/User Verification (UV) authentication model.

A **detailed breakdown** of this framework, including a detailed description of the CTAP protocol, can be **found** in the **work of Endler** [6], along with a more comprehensive insight into the topic of hardware security keys, including the cryptographic mechanisms that are commonly used.

2.2.1 Safety of CTAP over NFC

Since the protocol relies on **asymmetric cryptography** and challenge-response authentication, secret credentials are not transmitted directly over the communication channel, significantly reducing the risk of credential theft through passive eavesdropping. However, several attacks targeting FIDO/CTAP authenticators have been described.

Regarding NFC specifically, there is the possibility of **Client Impersonation (CI)** and **API Confusion (AC)** attacks. These methods and their results are described in detail by Casagrande and Antonioli [7]. The authors also suggest countermeasures to these attacks, among others, the most notable are different PIN for destructive commands, user interaction for user presence verification (i.e., pressing a button instead of only presenting the authenticator), and improved authenticator visual feedback.

Although the proposed countermeasures could be integrated without significant implementation complexity, the implementation presented in this thesis follows the CTAP specification and therefore does not introduce additional proprietary protection mechanisms beyond those required by the standard. It is worth noting that despite these exploits being published in 2023, none of the aforementioned methods has been introduced to the CTAP standard.

2.2.2 Post-Quantum Security of the CTAP Protocol

As quantum computing technology advances, it poses a significant **long-term threat** to the currently used public-key cryptographic schemes [8]. Although no quantum computer is currently capable of breaking them, the issue is important to acknowledge. Therefore, we provide a **brief examination** of the **post-quantum (PQ) security** of the CTAP protocol.

The version of CTAP implemented in this project (CTAP 2.2) **does not provide native PQ security**, as it relies on conventional public-key cryptographic primitives that could be compromised by sufficiently capable quantum adversaries. Nevertheless, the **current protocol architecture** is considered to be **PQ ready**. In particular, its PIN/UV authentication model requires active user verification and physical presence, while the protocol itself can be adapted to stronger cryptographic primitives without a fundamental redesign. Therefore, a full quantum-resistant operation **could be achieved** by replacing the current asymmetric mechanisms with post-quantum key encapsulation and signature mechanisms, provided that such changes are introduced consistently across both CTAP and WebAuthn within the FIDO2 framework [9]. The topic of making the CTAP protocol quantum-safe is discussed in more detail in work of **Kniep** [10].

Chapter 3
NFC

This chapter introduces the principles of **Near Field Communication (NFC)** relevant to the implementation presented in this thesis. Since NFC covers a broad set of radio technologies, operating modes, and higher-layer protocols, the discussion is **limited** primarily to **NFC-A**, **Card Emulation**, **ISO-DEP**, and **APDU**-based **communication**. Other notable technologies within the NFC ecosystem (as shown in Figure 3.2) are only briefly discussed to provide context and completeness.

3.1 Introduction to NFC

NFC is a short-range wireless communication technology derived from radio-frequency identification (RFID) systems. It enables **bidirectional data exchange** between devices in immediate or close proximity, typically within a range of up to 10 cm, with reliable operation occurring at distances of approximately 4 cm [11]. It is standardized primarily through ISO/IEC specifications and NFC Forum[1] specifications. The key specifications include ISO/IEC 14443, which defines proximity card communication (used in NFC-A and NFC-B), and ISO/IEC 18092, which specifies peer-to-peer communication. In addition, the NFC Forum defines higher-level protocols and data formats to ensure interoperability across devices.

NFC operates at a **carrier frequency (fc)** of **13.56 MHz** and is based on **inductive coupling** between **loop antennas**. In this mechanism, defined in [12], an active device generates an electromagnetic field, while a passive device modulates this field to transmit data. The technology supports three primary operating modes: **reader/writer mode, card emulation (CE) mode, and peer-to-peer mode**. From the perspective of this thesis, the **CE mode** is of particular importance, as it allows the device to behave as a contactless smart card and participate in ISO-DEP communication with an external reader.

3.2 Device Types

NFC devices fall into three categories: **active** devices (proximity coupling devices or PCDs), **passive** devices (proximity integrated circuit cards or **PICCs**), and **hybrid** devices (proximity extended devices or PXDs).

Active devices, or proximity coupling devices (**PCDs**), are equipped with their own power source and generate an electromagnetic field, enabling them to initiate and control NFC communication.

In contrast, **passive devices**, commonly implemented as **dynamic NFC tags**, do **not require an internal power supply**. Instead, they rely on the energy harvested from the

[1] `https://nfc-forum.org`

electromagnetic field generated by an active device (**Energy Harvesting**). **Communication** from the passive side is achieved through **load modulation**, i.e., by varying the impedance of the antenna circuit, which in turn modulates the carrier field. The characteristics of PICCs are defined in [13].

Hybrid devices, or proximity extended devices (**PXDs**), can operate in either PCD or PICC mode. Such devices may switch between modes automatically depending on the communication context, or allow **manual selection** by the user.

3.3 Device Roles

In NFC communication, two roles are distinguished according to which device initiates the interaction: the **initiator** and the **target**. These roles are defined at the protocol level and are independent of the specific hardware capabilities of the devices.

- **Initiator**
  The device that actively starts the communication and generates the initial RF field. It is responsible for polling and controlling the communication sequence, as is typically the case for an NFC reader.
- **Target**
  The device that responds to the initiator. Depending on the communication mode, it can passively modulate the initiator's field or actively generate its own signal. Typical examples include contactless smart cards, NFC tags, or peer devices.

It is important to distinguish these roles from the physical ability to generate an electromagnetic field. In passive communication mode, the initiator generates the RF field. In active communication mode, both devices are capable of generating their own RF field during their respective transmission phases.

NFC defines two communication modes based on how the electromagnetic field is generated and how the devices are powered: **active mode** and **passive mode**.

- **Active Mode**
  In active mode, both the initiator and the target are equipped with their own power sources and are capable of generating an RF field. Communication is performed through alternating transmission, where each device generates its own carrier field during its transmission phase. This mode is used primarily in peer-to-peer communication scenarios.
- **Passive Mode**
  In passive mode, only the initiator generates the RF field. The target may not have its own power source and may utilize energy harvesting. Data transmission from the target is achieved through load modulation of the existing carrier field. This mode is characteristic of typical NFC-A and NFC-B interactions, such as communication with contactless smart cards and NFC tags.

3.4 Operating Modes

The NFC Forum defines **three** primary **operating modes** that determine the role and behavior of a device within a communication session.

- **Reader/Writer Mode**
  In the reader/writer mode, the PCD operates as an initiator and interacts with passive PICCs. This mode is commonly used for applications such as reading NDEF records, accessing contactless cards, or configuring embedded devices.
- **Card Emulation Mode**
  In CE mode, the PICC behaves as a contactless smart card and acts as a target. It responds to commands issued by the PCD, typically using the ISO-DEP protocol. This mode is widely used in applications such as contactless payments, access control, and mobile wallets. Its main advantage is the possibility of implementing a more complex state machine while retaining the ability to utilize energy harvesting. This mode has two variants of implementation. It can either be implemented on a general-purpose MCU or using a dedicated **secure element (SE)** hardware, depending on the use case. Commercial NFC-capable security keys commonly rely on secure-element-based designs, as indicated, for example, by the Yubikey tech manual [14] or by the Google Titan documentation [15].
- **Peer-to-Peer Mode**
  The peer-to-peer mode enables half-duplex communication between two NFC-enabled devices, where both devices can alternately assume the roles of initiator and target. Communication in this mode is defined by [16] and the NFC Forum Logical Link Control Protocol (LLCP). It is primarily used for data exchange scenarios such as device pairing or file transfer, although its practical use has declined in favor of higher-throughput wireless technologies.

These operating modes build upon the previously described concepts of initiator and target roles, as well as active and passive communication modes. Their combination defines the functional behavior of NFC devices in real-world applications.

3.5 NFC RF Technologies

3.5.1 NFC-A

NFC-A is based on ISO/IEC 14443 Type A. In typical NFC-A communication, initialization and anticollision are performed at 106 kbit/s (fc/128). In theory, the specification allows speeds up to 27.12 Mbit/s (2fc) after activation.

Encoding and Modulation: Both NFC-A and NFC-B use **asymmetric** transaction methods. At 106 kbit/s, NFC-A employs **Modified Miller encoding** in the reader-to-card (PCD to PICC) direction, where logical bits are represented by pulse-position transitions in signal amplitude. The card-to-reader (PICC to PCD) direction uses **Manchester encoding** with a subcarrier at fc/16. These encodings improve robustness against noise and enable reliable clock synchronization.

The encoded signal modulates the RF carrier using different schemes depending on direction and data rate:

- **106 kbit/s to fc/16 (PCD to PICC):** 100% Amplitude Shift Keying (ASK) modulation with Modified Miller encoding where the carrier is fully suppressed during pause intervals.
- **106 kbit/s (PICC to PCD):** OOK (On-Off Keying) modulated subcarrier at fc/16 with Manchester encoding via load modulation.

- **fc/8 to fc/2**: Same modulation as NFC-B (see below).
- **3fc/4 to 2fc**: Phase Shift Keying (PSK) modulation, where the carrier phase shifts to encode information, reducing sensitivity to amplitude noise at higher speeds. ASK is retained for the PCD to PICC direction.

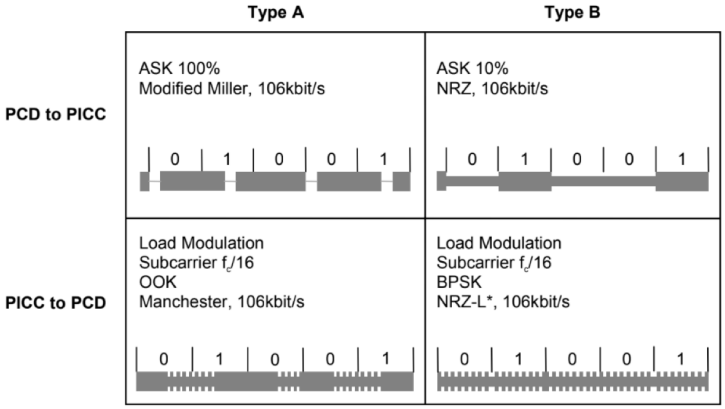

Figure 3.1. Examples of NFC-A and NFC-B modulation [12]

3.5.2 NFC-B

NFC-B is based on ISO/IEC 14443 Type B. Initialization and anticollision operate at 106 kbit/s, with a maximum data exchange speed of 6.78 Mbit/s (fc/2), lower than NFC-A's maximum.

Encoding and Modulation: NFC-B uses **NRZ-L (Non-Return-to-Zero Level)** encoding, where high and low amplitude levels directly represent logical 1 and 0 without returning to zero between bits.

- **PCD to PICC**: 10% ASK modulation. The carrier amplitude drops to approximately 90% of the amplitude to represent a logical 0, and remains at full amplitude for logical 1. The low modulation index keeps the carrier stable, reducing RF noise pickup.
- **PICC to PCD**: Binary Phase Shift Keying (BPSK) modulation of a subcarrier (fc/16), which allows the card to respond without being overwhelmed by the reader's transmitted signal. The initial phase of the subcarrier represents logical 1 and a 180 degree shift represents a logical 0.

The lower modulation index results in **improved robustness** against electromagnetic interference. This means that NFC-B is more suitable for applications requiring high reliability in noisy environments.

3.5.3 NFC-F

NFC-F is based on the JIS X 6319-4 specification and was originally developed by Sony under the commercial name FeliCa. Unlike NFC-A and NFC-B, NFC-F is not strictly derived from the ISO/IEC 14443 family, but instead follows a different protocol structure defined in Japanese industrial standards.

A key distinguishing feature of NFC-F is its higher data rates, typically 212 kbit/s or 424 kbit/s, which makes it suitable for latency-sensitive applications. The technology

is used primarily in Japan, particularly in transportation systems, contactless payment infrastructures, and access control solutions.

However, due to its regional focus and protocol differences, NFC-F is **not universally supported by all NFC readers**. Consequently, its applicability in general-purpose NFC implementations is limited and it is mentioned in this overview purely for completeness.

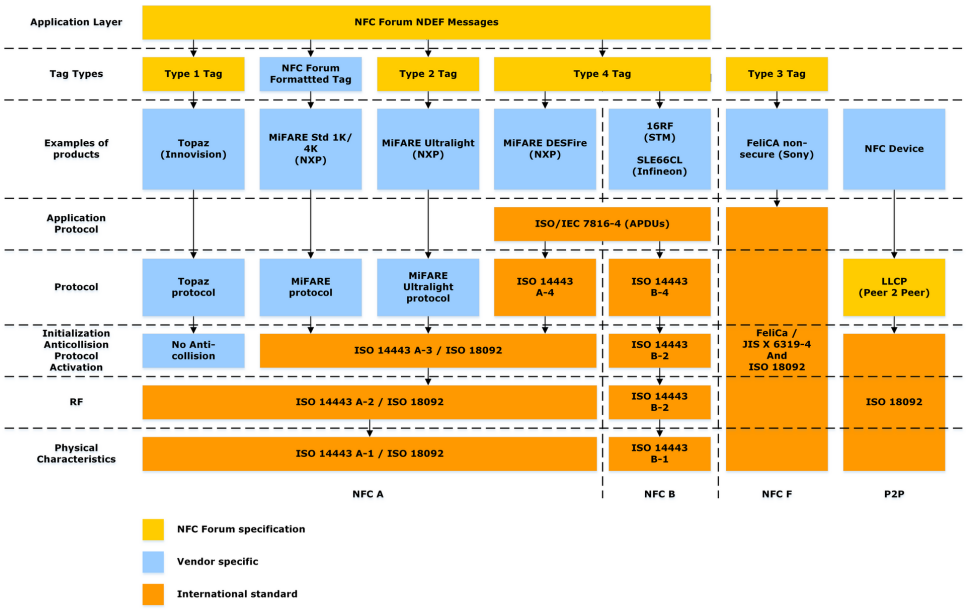

Figure 3.2. NFC Protocol Stack [17]

3.6 NFC-A State Machine

NFC-A anticollision and activation sequence has a defined state machine behavior, respecting [18]. This state machine must be respected for the correct initialization of the communication and protocol flow, as illustrated in Figure 3.3.

A PCD can use the following **commands** during the anticollision sequence an NFC-A PICC:

- **REQA** (REQuest, Type A)
  Broadcast command used to poll for any NFC-A cards in the field. Only cards in IDLE state respond.
- **WUPA** (Wake-UP, Type A)
  Similar to REQA, but also wakes cards in HALT state. Cards in both IDLE and HALT states respond.
- **ANTICOLLISION**
  Used to resolve collisions when multiple cards respond simultaneously, by iteratively isolating individual UIDs.
- **SELECT**
  Selects a specific card by its full UID, moving it into the ACTIVE (or ACTIVE*) state.
- **HLTA**
  Instructs the active card to enter the HALT state, suspending it until a WUPA command is received.

The NFC-A state machine has the following **states**:

- **POWER-OFF**
  The PICC is not in an RF field exceeding Hmin. Communication is not possible. The transition to IDLE occurs when the RF field strength exceeds Hmin.
- **IDLE**
  The PICC is powered by the RF field but has not yet been addressed. It responds only to REQA or WUPA commands. A valid REQA or WUPA transitions the card to READY.
- **READY**
  The PICC has responded to REQA or WUPA. The anticollision loop begins here: the PCD issues ANTICOLLISION commands to isolate the card's UID, then a SELECT command to uniquely address it. A successful SELECT transitions the card to ACTIVE. A failed SELECT or collision returns the card to IDLE.
- **READY***
  Functionally identical to READY, but entered exclusively from HALT via WUPA. A successful SELECT transitions the card to ACTIVE*.
- **ACTIVE**
  The PICC has been selected by its full UID and is ready to communicate. If the card supports ISO-DEP 3.7.1, it awaits a RATS command to enter the PROTOCOL state. Otherwise, it continues with its proprietary protocol. An HLTA command transitions the card to HALT.
- **ACTIVE***
  Functionally identical to ACTIVE, but entered from READY*. The distinction allows the PCD to identify cards that were reactivated from HALT, which may be relevant for session management. A HLTA command returns the card to HALT.
- **HALT**
  The PICC is suspended and ignores all commands except WUPA. This allows the PCD to selectively wake specific cards in a multi-card scenario. A valid WUPA transitions the card to READY*.
- **PROTOCOL**
  The PICC operates under the ISO-DEP protocol, initiated by a successful RATS/ATS exchange. All subsequent communication follows the negotiated parameters (FWT, FSC, etc.). The card remains in this state until the RF field is removed or a DESELECT command is issued, returning it to HALT or IDLE, respectively.

3.6.1 NFC-A Initialization and Anticollision

NFC-A uses a **bit-frame anticollision** method based on binary tree traversal. Since multiple PICCs may respond simultaneously to a REQA or WUPA command, their signals can overlap and corrupt each other, this is called a **collision**. The anticollision loop resolves collisions iteratively by narrowing down the UID of each card in the field. This overview **focuses solely on the NFC-A procedures**, although the standard also specifies behavior for hybrid tags supporting both the NFC-A and NFC-B types.

The commands defined in this subsection are later used in Chapter 5 to ensure correct operation of the NFC frontend.

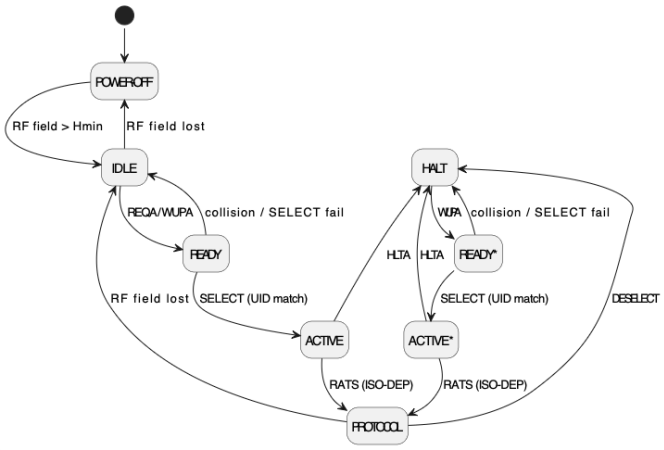

Figure 3.3. PlantUML diagram of the NFC initialization flow.

3.6.1.1 Polling

Before sending a REQA command, the PCD must present an unmodulated RF field for a minimum of **5.1 ms**. This ensures that passive PICCs, which rely on energy harvesting, are sufficiently powered before they are expected to respond. The PCD then broadcasts REQA to poll all Type A PICCs in IDLE state, or WUPA to additionally wake PICCs in HALT state.

3.6.1.2 Answer To reQuest (ATQA)

Upon receiving a valid REQA or WUPA, all eligible PICCs respond simultaneously with an **ATQA (Answer To reQuest, Type A)**, a 2-byte frame encoding (see Table 3.1:

- **Bit frame anticollision indicator** - which bit position is used as the anticollision entry point in the binary tree traversal
- **UID size indicator** - whether the UID is 4, 7 or 10 bytes, determining how many cascade levels the anticollision loop must traverse.

If multiple PICCs respond in the same slot, their ATQA frames collide at the bit level. The PCD detects this and proceeds with the anticollision loop to isolate individual cards.

Byte 1	Byte 2
Bit frame anticollision indicator	UID size indicator

Table 3.1. ATQA structure.

3.6.1.3 UID Structure and Cascade Levels

NFC-A PICCs have UIDs of three possible lengths, each requiring a different number of cascade levels to fully resolve:

- **Single (4 bytes)** - resolved in one cascade level (CL1)
- **Double (7 bytes)** - resolved across two cascade levels (CL1, CL2)
- **Triple (10 bytes)** - resolved across three cascade levels (CL1, CL2, CL3)

Each cascade level transmits 4 bytes of UID data at a time. When more levels remain, the first byte of the current level is set to the **Cascade Tag** (CT, value `0x88`), signaling to the PCD that the UID continues in a subsequent cascade level.

3.6.1.4 Anticollision Loop

The following sequence repeats for each required cascade level. Note that **bits are indexed from 1**.

- The PCD sends an **ANTICOLLISION** command (see Table 3.2) specifying the current cascade level (CL1 (0x93), CL2 (0x95), or CL3 (0x97)) and an **NVB (Number of Valid Bits)** field indicating how many leading bits of the UID are already known. Initially NVB = 0, meaning no bits are known yet.
- All PICCs in READY (or READY*) state whose UID matches the known prefix respond with their remaining UID bits.
- If exactly one PICC responds, no collision occurred and the PCD has resolved the full UID fragment for this level.
- If multiple PICCs respond simultaneously, a collision is detected at the first bit position where responses differ. The PCD appends either a **0** or **1** at the collision point, increments NVB, and reissues the ANTICOLLISION command - effectively traversing one branch of a binary tree. This repeats until a single PICC is isolated.
- Once the full 4-byte UID fragment for the current level is known, the PCD issues a **SELECT** command containing the complete fragment and a **BCC (Block Check Character)**, which is the XOR of the four UID bytes, used for integrity verification.
- The addressed PICC responds with a **SAK (Select AcKnowledge)**, confirming selection (Table 3.2). The SAK also indicates whether further cascade levels are required (bit 2 of SAK set) or whether the full UID has been resolved (bit 2 cleared).

SEL	NVB				UID CLn	BCC	E
CL	0	byte cnt.	0	bit cnt.	0–4 bytes (NVB)		

Table 3.2. ANTICOLLISION command structure.

Bit 2	Bit 5	Meaning
1	–	UID incomplete, further cascade levels required
0	0	UID complete, proprietary protocol
0	1	UID complete, ISO-DEP supported

Table 3.3. SAK bit 2 and bit 5 interpretation.

Once all cascade levels are resolved and the final SAK confirms a complete UID, the PICC transitions to **ACTIVE** state. If the SAK additionally indicates ISO-DEP support (bit 5 set), the PCD may proceed with a RATS command to initiate the ISO-DEP protocol. Otherwise, the PICC continues with its own proprietary protocol.

3.7 Communication Protocols

NFC communication is built on a layered protocol stack that defines how data is exchanged between devices after the initial activation phase. At the transport and higher communication layers, two primary protocols are used: ISO-DEP and NFC-DEP. In addition, several auxiliary protocol layers defined by the NFC Forum provide standardized data exchange formats.

3.7.1 ISO-DEP

The **ISO-DEP** (ISO Data Exchange Protocol), specified in [19], defines a transport layer for reliable data exchange between a PCD and a PICC. It extends the lower NFC-A/B protocols by introducing a **block-oriented, half-duplex communication mechanism** that enables structured and robust transmission of application data. This is the **most significant aspect** of the NFC technology for the subject of this thesis.

3.7.1.1 Activation and Parameter Negotiation

Before data exchange can begin, ISO-DEP requires an **activation phase** in which the initiator selects the target and negotiates communication parameters. This is performed using the **RATS** (Request for Answer To Select) and **ATS** (Answer To Select) mechanism.

First, the reader sends a RATS command (see Table 3.5). RATS is a 2-byte command consisting of a fixed command byte (E0) and a parameter proposal byte (PP).

RATS consists of a fixed command byte `0xE0` followed by a parameter byte. The upper nibble of the parameter byte encodes the Frame Size for Device Integer (FSDI); the conversions from FSDI to FSD are shown in Table 3.4. The lower nibble of RATS encodes the Card Identifier (CID), which logically addresses a specific card when multiple cards are in the field. This value remains assigned to the PICC until it leaves the field. CID=15 is reserved.

FSDI	0	1	2	3	4	5	6
FSD (bytes)	16	24	32	40	48	64	96
FSDI	7	8	9	A	B	C	D–F
FSD (bytes)	128	256	512	1024	2048	4096	RFU

Table 3.4. FSDI to FSD conversion table

E0	Parameter Proposal		CRC1	CRC2
1B	FSDI	CID	1B	1B

Table 3.5. RATS structure.

The ATS response consists of a total length value (TL), format byte (T0), optional interface bytes (TA, TB, TC) and optional Historical Bytes (up to 15 bytes). These

TL	T0	TA(1)	TB(1)	TC(1)	T1	T_k	CRC1	CRC2
1B	1B	interface			1B	HB	1B	1B

Table 3.6. ATS structure.

fields define key communication parameters, including timing constraints and frame sizes. All blocks are terminated by a two-byte CRC. This is illustrated in Table 3.6.

The interface bytes can contain these values:

- **Frame Size (FSC/FSD)**
  Defined by the FSCI field in T0, it specifies the maximum payload size of a single frame for the PICC (FSC) and the PCD (FSD), determining whether chaining is required.
- **Frame Waiting Time (FWT)**
  Specifies the maximum time the initiator waits for a response from the target. It is derived from the Frame Waiting Time Integer (FWI) in TB(1) using the formula:

$$FWT = \frac{256 \times 16}{f_c} \times 2^{FWI}$$

- **Frame Guard Time (FGT)**
  Derived from the Guard Time Integer (GTI) in TB(1), FGT specifies the minimal delay that must elapse before next transaction. This prevents receiver saturation and ensures reliable synchronization.
- **Device Identifier (CID)**
  When there are multiple cards present in the field, the initiator can use CID (0–14), specified in TC(1), to address a specific card. If not used, the CID is typically set to 0 or completely omitted.
- **Node Address Support (NAD)**
  TC(1) may also indicate support for the optional NAD mechanism used for logical node addressing at higher protocol layers. In practice, NAD is rarely implemented in modern NFC devices.
- **Historical Bytes (HB)**
  Optional data included in the ATS; they are to be interpreted according to the card manufacturer's specifications, and they may provide additional information about the capabilities of the card if present. Contents of HB are further specified in [20].

3.7.1.2 Data Exchange Blocks

Communication in **ISO-DEP** is organized into protocol data units called **blocks**, which are exchanged between the initiator and the target. There are two types of blocks: **Standard** and **Extended** blocks.

The general structure of ISO-DEP blocks is illustrated in Tables 3.7 and 3.8. Note that fields in brackets (`[]`) are optional.

Each block begins with a mandatory one-byte **PCB (Protocol Control Byte)**, which identifies the block type and encodes protocol control information. Depending on the block

Prologue			Information	Epilogue	
PCB	[CID]	[NAD]	[INF]	ECD	
1B	1B	1B	application data	CRC1	CRC2

Table 3.7. Structure of a Standard ISO-DEP block

Length	Prologue			Information	Epilogue
LEN	PCB	[CID]	[NAD]	[INF]	ECD
2B	1B	1B	1B	application data	CRC (4B)

Table 3.8. Structure of an Extended ISO-DEP block

type, the PCB may contain information such as block numbering, chaining indication, acknowledgment status, or supervisory commands. Depending on the negotiated protocol configuration, the blocks can also contain optional **CID** and **NAD**. I-Blocks and some S-Blocks may also include an **INF** field that contains application or supervisory data.

Based on the two leftmost bits of PCB (see Table 3.9), we recognize three types of ISO-DEP blocks:

- **I-Blocks** (Information blocks)
  Used to transport application data (including APDUs), these are fragmented into I-Blocks and reassembled upon reception.
- **R-Blocks** (Receive Ready blocks)
  Used for acknowledgments and retransmission control.
- **S-Blocks** (Supervisory blocks)
  Used for protocol management and control operations, such as waiting time extension (WTX) or deselection (transaction termination).

b8	b7	Block Type
0	0	I-Block
0	1	RFU
1	0	R-Block
1	1	S-Block

Table 3.9. ISO-DEP block type encoding

The S-Blocks with WTX are especially important for blocking operations like cryptographic calculations that are an essential part of LionKey. It is important to handle WTX in time in order to avoid timeouts.

3.7.1.3 APDU

ISO-DEP serves as the transport layer for **Application Protocol Data Units (APDUs)**, which implement a command–response communication model. APDUs are a standard-

ized data format used in smart card systems for structured information exchange defined by ISO/IEC 7816-4.

In this model, the initiator sends a **command APDU (C-APDU)**, and the target responds with a **response APDU (R-APDU)**. This abstraction allows higher-level protocols, including smart card applications and CTAP over NFC, to operate independently of the underlying transmission mechanism while relying on ISO-DEP for reliable delivery.

A C-APDU consists of a **header** (CLA, INS, P1, P2) and an **optional payload** defined by the fields Lc (length of command data) and Le (expected length of response data).

APDUs can be encoded in short or extended format. In the short format, the length fields are limited to one byte, allowing payloads of up to 255 bytes. The extended format uses two-byte length fields, enabling payloads of up to 65,535 bytes.

In the short format, the C-APDU (see Table 3.10) consists of the mentioned 4-byte header followed by the optional fields Lc, Data (0–255 bytes) and Le, where both Lc and Le are encoded using a single byte.

In the extended format (see Table 3.11), the presence of the leading `0x00` byte indicates extended length encoding, followed by two-byte fields for Lc and Le. The exact position of Le depends on the APDU case.

CLA	INS	P1	P2	[Lc]	[Data]	[Le]
1 byte	1 byte	1 byte	1 byte	0/1	0–255	0/1

Table 3.10. Byte-level structure of short C-APDUs.

CLA	INS	P1	P2	00	Lc(Hi)	Lc(Lo)	Data/Le
1B	1B	1B	1B	ext. marker	length		variable

Table 3.11. Byte-level structure of extended C-APDUs.

A R-APDU may contain **response data** and is always terminated by a two-byte **status word** (SW1, SW2), indicating the result of the operation.

Data	SW1	SW2
(optional) response data	status	word

Table 3.12. General structure of a R-APDU.

3.7.1.4 APDU Communication Example

An example of APDU communication can be seen in Figure 3.4.

```
CE: start waiting for command
CE: negotiated max frame size: 252
CE: start RX successful
Received APDU (13 bytes):00A4040008A0000006472F0001
NFC: APDU parsed successfully
NFC: FIDO AID selected
Sent APDU (10 bytes): 4649444F5F325F309000
```

Figure 3.4. Device log of processing an APDU and selecting the FIDO applet.

```
C-APDU: 00  A4  04 00 08 A0 00 00 06 47 2F 00 01
        |   |   |  |  |  \_______ Data ________/
       CLA INS P1 P2 Lc

R-APDU: 46 49 44 4F 5F 32 5F 30   90 00
        \_______ Data ________/ \SW1 SW2/
```

In this example, we can see the byte structure of a C-APDU, sent by a PCD, followed by an R-APDU, sent by a PICC. The C-APDU consists of the following fields: CLA = `0x00` (interindustry class byte - a standard entry point for commands), INS = `0xA4` (SELECT instruction), P1 = `0x04` (selection by name) and P2 = `0x00`. The field Lc = `0x08` specifies the length of the data field, which contains the 8-byte FIDO **application identifier (AID)** (`A0 00 00 06 47 2F 00 01`). After processing this message, the device responds with an R-APDU.

The R-APDU contains the response data followed by the status word. The data field corresponds to the ASCII string “FIDO_2_0”, while the status word `0x9000` indicates successful execution of the command.

3.7.1.5 Chaining Mechanisms

Within ISO-DEP, APDUs are transported using I-Blocks. Since the maximum size of a single ISO-DEP frame is limited by the negotiated frame size, APDUs exceeding this size must be fragmented. This is achieved through **ISO-DEP chaining**, where a single APDU is split across multiple I-Blocks and reassembled by the receiving side.

In addition to transport-level fragmentation, large responses may also be handled at the application level. In the case of short APDUs, the response length is limited by the value of the Le field. If the response exceeds this limit, it can be delivered in multiple steps using the status word mechanism (e.g., SW1 = `0x61`), where the client issues the GET RESPONSE command to retrieve the remaining data.

The two chaining mechanisms operate at different layers: ISO-DEP chaining ensures reliable transmission of a single APDU over multiple frames, while application-level response handling enables large response data to be retrieved through multiple APDU commands, especially when short APDU length fields are used.

3.7.1.6 NDEF

The **NFC Data Exchange Format (NDEF)** is a standardized binary message format used to encode and exchange application-level data between NFC devices and tags. NDEF operates above the transport and protocol layers and is independent of the underlying RF technology.

An **NDEF Message** consists of one or more **NDEF Records**. Each of those records consists of a 6-9 bytes long header and a payload. The type of the record is identified by the **Type Name Format (TNF)** field on the first byte of the header. TNF determines how the payload should be interpreted. Payload types include plain text, URIs, MIME-typed data (i.e. images), and application-defined external types identified using a domain-name-based naming convention. NDEF is widely used in the reader/writer mode and is broadly supported across NFC-capable devices and mobile operating systems.

Due to its widespread adoption as a standard mechanism for application-level NFC data exchange, NDEF can be considered a fundamental component of practical NFC interoperability. Therefore, in order to provide a more complete implementation of basic NFC functionality as given by the guidelines, we also made basic NDEF support a part of the implementation scope alongside CTAP communication.

In this implementation, we access NDEF records via ISO-DEP using APDU-based communication. The communication is as follows: The reader first selects the NDEF application using its AID, then accesses the **Capability Container (CC)** file to determine the tag's capabilities and supported data sizes. Finally, the reader selects the NDEF data file itself and performs read or write operations.

APDU commands for those operations are:

- **SELECT** (by AID or file identifier)
  Selects the NDEF application or a specific file within it.
- **READ BINARY**
  Reads the contents of the currently selected file, typically used to retrieve the NDEF message.
- **UPDATE BINARY**
  Writes data into the currently selected file, allowing the stored NDEF message to be modified if write access is permitted.

3.7.2 NFC-DEP

The **NFC-DEP** (NFC Data Exchange Protocol), defined in [16], is designed for peer-to-peer communication between two **active** NFC devices. Unlike ISO-DEP, which follows a reader–card model, NFC-DEP enables symmetric communication, allowing both devices to alternate between initiator and target roles.

In contrast to ISO-DEP, NFC-DEP **is not based on the APDU command–response model** used by ISO-DEP. Instead, communication is based on **message-oriented data exchange**, making it more suitable for peer-to-peer scenarios rather than card emulation.

Since CTAP over NFC relies on APDU communication over ISO-DEP, NFC-DEP is outside the scope of this implementation.

Chapter 4
Implementation

In this chapter, the **technologies used** in the implementation of the NFC transport layer for the LionKey security key as well as the **software implementation** are described. The reasons for system design decisions come mainly from facts stated in Chapter 3. As stated in Chapter 1, the aim of this thesis is to **implement NFC as the transport layer** for the already implemented security key. The implementation scope is highlighted in Figure 4.1. Our implementation follows the **FIDO CTAP2.1 standard** [21].

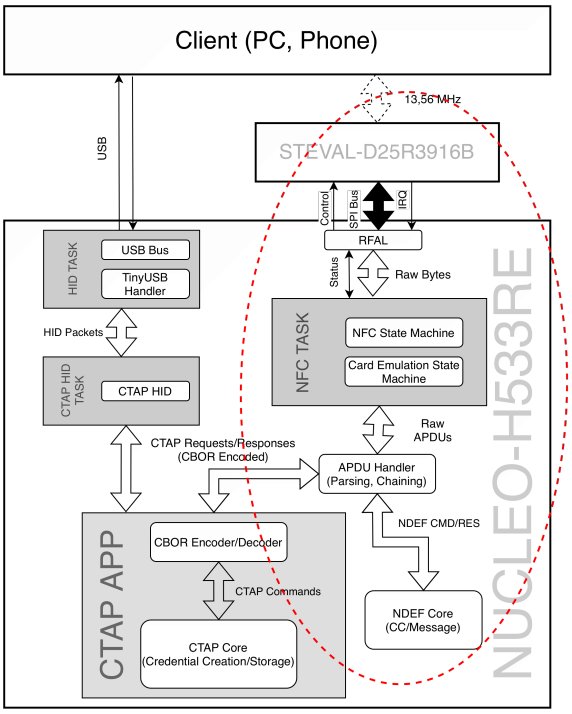

Figure 4.1. Diagram of the LionKey Stack (implementation scope circled in red).

4.1 NFC technologies used

After evaluating the available NFC operating modes and specifications, it becomes evident that the Card Emulation Mode is the only mode compatible with the intended use case. The objective is for the device to behave as a passive entity, analogous to a contactless smart card, rather than as an active initiator.

In the ideal case, SE-based card emulation would be used. However, this is not feasible with respect to the existing LionKey hardware architecture. This approach requires the a of a dedicated SE capable of isolated execution and secure key storage, which is

not available in the current design. Consequently, the implementation must be based on host-based card emulation, where all protocol handling is performed by the microcontroller (MCU).

The required functionality is achieved most appropriately through the emulation of a Type 4 NFC Forum Tag. In contrast to simpler tag types, Type 4 Tag operation is based on the ISO-DEP protocol and supports structured command–response communication via APDUs. This enables the implementation of a state machine and allows for computation within the tag (see 3.2). NFC-A was selected over NFC-B for the implementation due to its greater support across mobile operating systems.

4.2 Hardware

The hardware chosen for the implementation is the **STEVAL-D25R3916B**, a daughter board of the STEVAL-25R3916B platform manufactured by STMicroelectronics [22]. This kit is an evaluation board for the 25R3916B NFC chip, which turned out to be the best option for the purpose of this thesis.

The 25R3916B chip belongs to the STMicroelectronics universal NFC device family. It supports all main NFC operating modes, including reader/writer mode, peer-to-peer communication, and card emulation; however, it does not provide support for SE–based card emulation.

Unlike dedicated NFC tag ICs with fixed memory structures, the chip operates as a low-level RF frontend, exposing frame-level control to the application layer. Thanks to this, it enables software-driven card emulation, allowing the host microcontroller to implement the protocol logic.

4.2.1 Board Specifications

The STEVAL-D25R3916B embeds the **ST25R3916 chip** with an antenna etched on the PCB and the related Very High Bit Rate (VHBR) tuning circuit, allowing it to communicate at higher speeds.

In terms of host communication, the board provides both Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) interfaces, complemented by an interrupt request line (IRQ) used to signal events to the MCU.

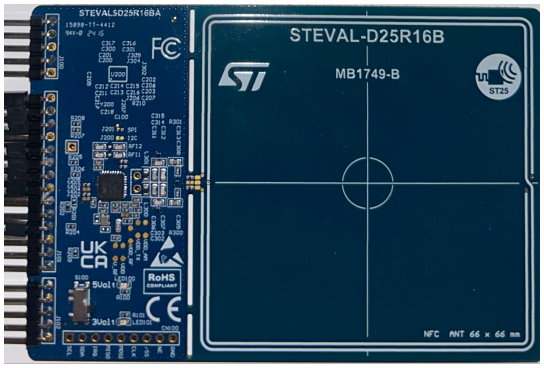

Figure 4.2. The STEVAL-D25R3916B board.

4.2.2 Note: Energy Harvesting

A significant limitation of the selected device is its absence of energy harvesting capabilities. Consequently, the security key cannot operate as a fully passive, standalone authenticator and requires an external power source during operation. It is important to note that achieving true standalone functionality would require not only an NFC frontend capable of efficient energy harvesting but also a substantially lower-power MCU.

The power demands of the current hardware platform exceed what can be reliably supplied by the electromagnetic field generated by typical PCDs, making passive operation impossible under these conditions.

4.3 Middleware

The middleware used in this project is the STSW-ST25RFAL002 package, provided by STMicroelectronics as an implementation of the RFAL (RF Abstraction Layer) software for the ST25R3916B NFC frontend. RFAL represents a hardware-oriented abstraction layer that encapsulates low-level device control and RF protocol handling. It provides a set of primitives for device initialization, RF configuration, register-level access, and data exchange.

In our implementation, RFAL forms the intermediary layer between the LionKey application and the NFC frontend. The application exchanges data and control requests with RFAL, while RFAL is responsible for direct interaction with the NFC device, low-level NFC control, communication parameters negotiation, subsequent ISO-DEP R-APDU fragmentation (i.e. when the size APDU being sent is larger than the negotiated frame size) and meeting FWT deadlines (by automatically sending WTX blocks). The higher-level communication logic, including the NFC state machine and protocol flow control, including short APDU chaining, is implemented in the application layer.

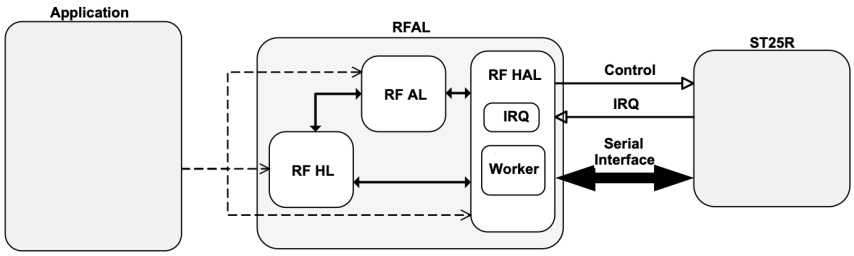

Figure 4.3. Communication flow between the Application (LionKey) and the NFC device using RFAL [23].

4.4 Communication with the NFC Frontend

This section describes the communication interface between the MCU and the NFC frontend. Full connection scheme is described in Figure 4.4.

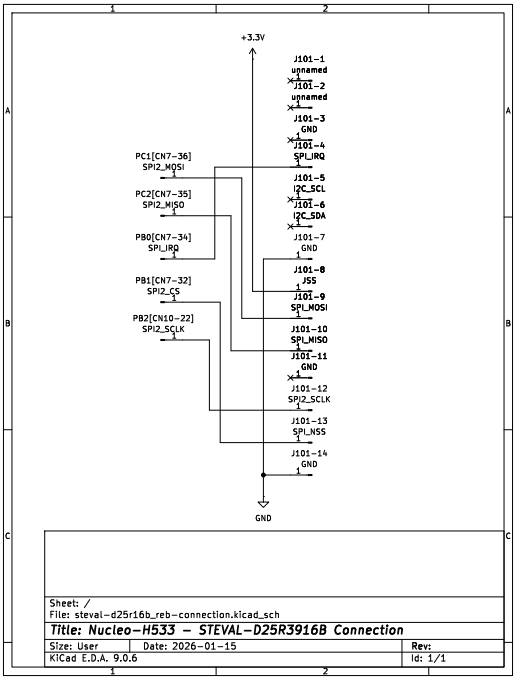

Figure 4.4. KiCad scheme showing how the STEVAL-D25R3916B board is connected to the Nucleo-H533 MCU.

4.4.1 SPI

The communication between the MCU and the NFC frontend is implemented using SPI. Compared to I2C, SPI provides higher data throughput, lower communication latency, and, most importantly, fully deterministic timing due to its synchronous nature and absence of bus arbitration. These properties are critical to ensure protocol compliance. Any excessive delay in data transfer between the front end and the MCU may lead to FWT violations, thus prematurely terminating the NFC session.

In our implementation, SPI operates in a master–slave configuration, with the MCU acting as the master and ST25R3916B acting as the slave device. Chip select (CS) is manually driven via a GPIO pin to ensure precise control over transaction boundaries. The specifics of the connection are illustrated in Figure 4.4.

4.4.2 Interrupt Handling

The IRQ line is asserted by the NFC frontend to indicate events such as reception of RF frames, completion of transmission, state changes in the RF field or error conditions.

Upon receiving the interrupt, the MCU reads the corresponding status registers through SPI and sends the event to RFAL. This mechanism enables prompt processing of incoming ISO-DEP frames and immediate reaction to transmission errors.

4.5 Software Implementation

In terms of code, the software implementation follows the practices set in the original project. It is written in C with no dynamic memory allocations utilizing a layered design that separates the NFC transport layer, APDU processing, and higher-level FIDO logic.

The implementation follows the NFC Forum Type 4 Tag specification and supports both CTAP2-related communication and basic NDEF communication, thus fulfilling the goal of the thesis of "Demonstrating basic NFC functionality".

The NFC functionality is implemented using a set of state machines integrated into the existing application architecture, with the NFC worker function being run periodically from the main application loop.

4.5.1 NFC State Machine

The primary state machine models the operational state of the NFC frontend and distinguishes between the following states (refer to Figure 4.5):

- **Uninitialized (NOTINIT)**
  Initial state of the device, waiting for initialization. Transitions to the DISCOVERY state after initialization.
- **Inactive (Idle) state (DISCOVERY)**
  The device is ready but not within the range of a PCD . Transitions to the CE_ACTIVE state when the ISO-DEP initialization sequence is done (see 3.6).
- **Active state (CE_ACTIVE)**
  CE mode is active and data exchange with the PCD is in progress. The transition to START_DISCOVERY state commences when an error occurs, the device is requested to go idle by the PCD, or when the device leaves the PCD field.
- **Reset state (START_DISCOVERY)**
  The communication session has ended and the NFC runtime context is reinitialized to its default configuration. The device then immediately transitions to the DISCOVERY state.

4.5.2 CE State Machine

To ensure non-blocking operation and enable concurrent execution of other critical components in parallel with NFC, the card emulation task is implemented as a separate Card Emulation state machine (within NFC_CE_ACTIVE in 4.5). This secondary state machine governs the progression of individual NFC transactions, including reception and transmission phases, and consists of the following states:

- **Idle (CE_STATE_IDLE)**
  The card emulation layer is activated but no exchange has been initiated yet. Initializes context. Transitions to CE_STATE_WAIT_RX when communication is initialized.
- **Wait RX (CE_STATE_WAIT_RX)**
  The device is listening for an incoming APDU from the PCD. The RFAL exchange status is polled until a complete frame is received. Upon receiving a complete frame, it transitions to PROCESS_RX. In case sleep is requested by the PCD, the state machine goes to IDLE. Otherwise, if an unexpected error occurs (i.e. an incomplete frame is received), it transitions to ERROR_RECOVERY.

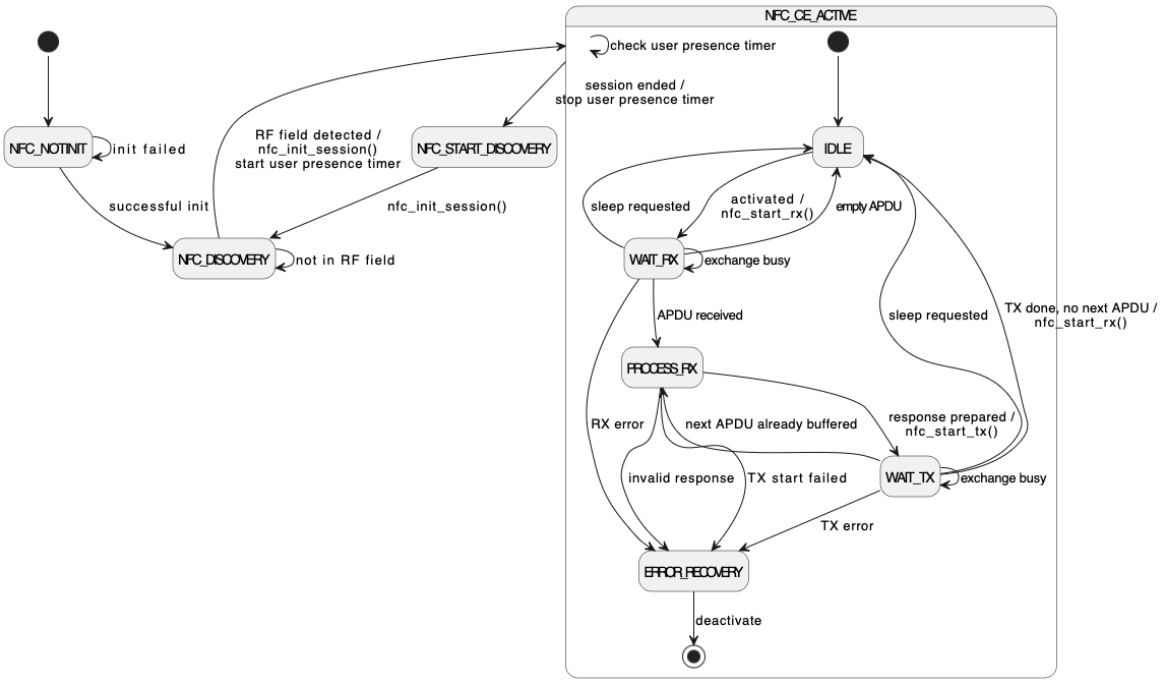

Figure 4.5. PlantUML diagram of the implemented state machine.

- **Process RX (CE_STATE_PROCESS_RX)**
  The received APDU is parsed and a response is constructed. Upon success, transmission is initiated and the state advances to WAIT_TX.
- **Wait TX (CE_STATE_WAIT_TX)**
  The device waits for the transmission of the response to complete, after which the state returns to WAIT_RX to await the next command.
- **Error Recovery (CE_STATE_ERROR_RECOVERY)**
  An unrecoverable protocol error has occurred. The session is terminated, and the primary state machine transitions to NFC_START_DISCOVERY to reinitialize the context for a new session.

4.5.3 Managing the CE Machine

Management of the communication state machine is performed by periodic polling of the exchange status provided with the RFAL library. Although an interrupt-driven approach would normally be more suitable in an embedded device, we opted to follow the same coding practices as the original project. Therefore, polling is used for transaction handling instead of interrupts. The IRQ line is, however, used internally by the RFAL for its own state machine handling.

```
static bool nfc_handle_wait_rx(void)
{

    if (nfc_poll_transmission_status(RX) == false)
    {
        return false;
    }
    ...
}
```

```
static bool nfc_poll_transmission_status(transmission_line_t line)
{
    const ReturnCode err = rfalNfcDataExchangeGetStatus();

    /* Transaction still in progress */
    if (err == RFAL_ERR_BUSY)
    {
        return false;
    }

    /* Sleep requested by the PCD */
    if (err == RFAL_ERR_SLEEP_REQ)
    {
        nfc_runtime.ce_state = CE_STATE_IDLE;
        return false;
    }

    /* Other unexpected error, enter error recovery state */
    if (err != RFAL_ERR_NONE)
    {
        nfc_enter_error_recovery(err);
        return false;
    }

    return true;
}
```

Upon receiving an APDU from the PCD (PROCESS_RX), the message is parsed and processed according to its semantics. The parser supports all defined APDU formats and handles edge cases (i.e., incorrect length or unsupported CLA) according to [20].

```
static bool nfc_handle_process_rx(void)
{
  nfc_runtime.tx_len = nfc_parse_and_respond(&nfc_runtime.ce_ctx,
                                             nfc_runtime.rx_data,
                                             *nfc_runtime.rcv_len,
                                             nfc_runtime.tx_buf,
                                             sizeof(nfc_runtime.tx_buf));

    if ((nfc_runtime.tx_len == NFC_PARSE_WRONG_SIZE)
         || (nfc_runtime.tx_len == 0))
    {
        nfc_enter_error_recovery("APDU response invalid", 0);
        return false;
    }

    if (!nfc_start_tx(nfc_runtime.tx_buf, nfc_runtime.tx_len))
    {
        rfalNfcDeactivate(RFAL_NFC_DEACTIVATE_DISCOVERY);
    }
```

```
    return false;
}
```

The current context of the application transmission state is managed by the t4t_context struct which hold the context of the current NFC session.

```
typedef struct
{
    uint8_t selected_file;
    /*!< currently selected file */
    uint8_t selected_app;
    /*!< currently selected applet */
    const uint8_t *cc_file;
    /*!< contents of the CC file */
    uint16_t cc_file_len;
    /*!< length of the CC file */
    uint8_t *ndef_file;
    /*!< contents of the NDEF file */
    uint16_t ndef_file_len;
    /*!< length of the NDEF file */
    uint16_t fid_cc;
    /*!< file ID of the CC file */
    uint16_t fid_ndef;
    /*!< file ID of the NDEF file */
    bool ndef_write_allowed;
    /*!< flag indicating whether the NDEF file is writable */

    /*! Chaining helpers */
    uint8_t chain_buf[CTAPHID_MAX_PAYLOAD_LENGTH];
    /*!<buffer for building responses larger than Le */
    uint16_t chain_offset;
    /*!<offset in the chain buffer for the current response being built*/
    uint16_t chain_len;
    /*!<remaining length of the response to be sent in the chain buffer*/

    bool chaining_in;
    bool chaining_out;
    /*!<is chaining in progress */
    bool extended;
    /*!<is extended apdu being used in this context*/
    size_t resp_len_expected;
    /*!<expected response length */
} t4t_context_t;
```

4.5.4 APDU Processing

The incoming APDU is processed as follows:

```
uint16_t nfc_parse_and_respond(t4t_context_t *ctx,
                               uint8_t *rx_data,
                               uint16_t rx_data_len,
                               uint8_t *tx_buf,
```

```
                                uint16_t tx_buf_len )
{
    nfc_apdu_t apdu;
    apdu_parse_status_t err;

    ...
    /* 1. - parse APDU */
    err = nfc_parse_apdu(rx_data, rx_data_len, &apdu);

    /* 2 - handle deselect */
    if (apdu.ins == NFC_INS_DESELECT) {
        ctx->selected_app  = APP_NONE;
        ctx->selected_file = FILE_NONE;
        reset_chain();
        return nfc_put_sw(tx_buf, NFC_SW_OK);
    }

    /* 3. - handle handshake */
    if ((apdu.cla == NFC_CLA_ISO) &&
        (apdu.ins == NFC_INS_SELECT) &&
        (apdu.p1  == 0x04U) &&
        (apdu.p2  == 0x00U))
    {
        ...
        /* FIDO selected (as per 11.3.3. Applet selection) */
        if ((apdu.lc == sizeof(FIDO_AID)) &&
            (memcmp(apdu.data, FIDO_AID, sizeof(FIDO_AID))) == 0)
        {
            ...
            return nfc_put_sw(tx_buf, NFC_SW_OK);
        }

        /* NDEF selected, proceed with handshake */
        if ((apdu.lc == sizeof(NDEF_AID)) &&
            (memcmp(apdu.data, NDEF_AID, sizeof(NDEF_AID))) == 0)
        {
            ...
            return nfc_put_sw(tx_buf, NFC_SW_OK);
        }

        /* Unknown request */
        ...
        return nfc_put_sw(tx_buf, NFC_SW_FILE_NOT_FOUND);
    }

    /* 4. - Handle the request in the context of the selected app */
    switch (ctx->selected_app)
    {
        case APP_FIDO:
```

```
            /* Handle short/extended APDU or chaining request */
            ...
        case APP_NDEF:
           ...
        default:
            return nfc_put_sw(tx_buf, NFC_SW_COND_NOT_SATISFIED);
    }
}
```

4.5.5 APDU Chaining

CTAP2 short-APDU responses that exceed the NFC frame size are split across multiple APDUs using chaining (note that we are referring to APDU chaining, not ISO-DEP chaining which is done internally by RFAL), as requested by [24]. There are **two types of chaining** in the CTAP protocol (although they are not named in the CTAP specification) - chaining in the incoming direction (let us call this **in-chaining**) and in the outcoming direction (**out-chaining**).

The CTAP specification states that "If the request was encoded using short APDU encoding, the authenticator MUST respond using ISO 7816-4 APDU chaining". The in-chaining has its own CLA byte - 0x90, marking the first short APDU in the incoming chain, the subsequent APDU(s) come with a CLA byte of 0x80. Out-chaining, as mentioned, follows [20].

The implementation handles **in-chaining** in the following way:

```
static uint16_t fido_handle_ctap_chain_in(t4t_context_t *ctx,
                                          const nfc_apdu_t *apdu,
                                          uint8_t *tx_buf,
                                          uint16_t tx_buf_len)
{
    // first apdu chain command, start chaining
    if (apdu->cla == NFC_CLA_CTAP_CHAIN)
    {
        if (apdu->lc > sizeof(ctx->chain_buf)) {
            reset_chain(ctx);
            return nfc_put_sw(tx_buf, NFC_SW_WRONG_LENGTH);
        }
        memcpy(ctx->chain_buf, apdu->data, apdu->lc);
        ctx->chain_len = (uint16_t)apdu->lc;
        ctx->chain_offset = 0U;
        ctx->chaining_in = true;
        return nfc_put_sw(tx_buf, NFC_SW_OK);
    }

    if (apdu->cla != NFC_CLA_CTAP)
    {
        return nfc_put_sw(tx_buf, NFC_SW_NOT_FOUND);
    }

    // subsequent apdu chain command, append to chain buffer
```

```
    if ((size_t)ctx->chain_len + apdu->lc > sizeof(ctx->chain_buf))
    {
        // protect against overflow
        reset_chain(ctx);
        return nfc_put_sw(tx_buf, NFC_SW_WRONG_LENGTH);
    }

    memcpy(&ctx->chain_buf[ctx->chain_len], apdu->data, apdu->lc);
    ctx->chain_len += (uint16_t) apdu->lc;

    if (!apdu->has_le)
    {
        // more chaining to come
        return nfc_put_sw(tx_buf, NFC_SW_OK);
    }
    // last chain command, process the full request
    ctx->chaining_in = false;
    ctx->resp_len_expexted = (apdu->le == 0U) ?
                        NFC_APDU_SHORT_MAX_LEN : (uint16_t)apdu->le;
    return fido_handle_ctap(ctx,
                            ctx->chain_buf,
                            ctx->chain_len,
                            tx_buf,
                            tx_buf_len);
}
```

Out-chaining is first handled in the CTAP command processing function:

```
static uint16_t fido_handle_ctap(t4t_context_t *ctx,
                                 const uint8_t *data_in,
                                 size_t data_in_len,
                                 uint8_t *tx_buf,
                                 uint16_t tx_buf_len)
{
    ...
    /* status + CBOR */
    size_t   full_len = 1U + nfc_ctap_response.length;
    ...
    /* short, check need for chaining */
    if (full_len <= ctx->resp_len_expexted) {
        reset_chain();
        return nfc_build_response(nfc_ctap_response_buffer,
                                  full_len,
                                  NFC_SW_OK,
                                  tx_buf,
                                  tx_buf_len);
    }

    if (full_len > sizeof(ctx->chain_buf))
    {
        /* Protect against oversized response */
```

```
        ctx->chain_len = 0U;
        return nfc_put_sw(tx_buf, NFC_SW_WRONG_LENGTH);
    }

    const uint16_t first_chunk = (full_len <= ctx->resp_len_expexted) ?
                          (uint16_t)full_len : ctx->resp_len_expexted;

    memcpy(ctx->chain_buf, nfc_ctap_response_buffer, full_len);
    ctx->chain_offset = first_chunk;
    ctx->chain_len    = (uint16_t)(full_len - first_chunk);
    ctx->is_chaining  = true;

    uint8_t remaining = (ctx->chain_len > 0xFFU) ?
                         0xFFU : (uint8_t)ctx->chain_len;
    uint16_t chain_sw = (uint16_t)(NFC_SW_CHAIN | remaining);

    return nfc_build_response(ctx->chain_buf,
                              first_chunk,
                              chain_sw,
                              tx_buf,
                              tx_buf_len);
}
```

And the subsequent chunks are sent after receiving a CTAP C-APDU with the `GET_RESPONSE (0xC0)` INS:

```
static uint16_t fido_handle_ctap_chain_out(t4t_context_t *ctx,
                                           const nfc_apdu_t *apdu,
                                           uint8_t *tx_buf,
                                           uint16_t tx_buf_len)
{

    /* no chain, return SW error */
    if (ctx->chain_len == 0U)
    {
        return nfc_put_sw(tx_buf, NFC_SW_COND_NOT_SATISFIED);
    }

    const uint16_t le      = (apdu->le == 0U) ?
                              NFC_APDU_SHORT_MAX_LEN: apdu->le;
    const uint16_t to_send = (ctx->chain_len <= le) ?
                              ctx->chain_len : le;

    /* get position */
    uint8_t *chunk = &ctx->chain_buf[ctx->chain_offset];

    /* if this chunk is the last one, send OK status word */
    if ((ctx->chain_len - to_send) == 0U)
    {
        /* Last chunk */
        reset_chain(ctx);
```

```
        return nfc_build_response(chunk,
                                  to_send,
                                  NFC_SW_OK,
                                  tx_buf,
                                  tx_buf_len);
    }

    ctx->chain_offset += to_send;
    ctx->chain_len    -= to_send;

    const uint8_t remaining =(ctx->chain_len >= NFC_APDU_SHORT_MAX_LEN) ?
                              0xFFU : (uint8_t)ctx->chain_len;
    const uint16_t chain_sw = (uint16_t)(NFC_SW_CHAIN | remaining);

    return nfc_build_response(chunk,
                              to_send,
                              chain_sw,
                              tx_buf,
                              tx_buf_len);
}
```

4.5.6 NDEF Handling

The NDEF state machine exposes a static NDEF file that contains a single URI record that directs the user to the lionkey.dev website. The NDEF file is structured as follows:

```
static const uint8_t NDEF_FILE[] = {
0x00, 0x10, // NLEN (16 bytes)
0xD1, // NDEF Record Header
0x01, // TYPE LENGTH = 1 byte
0x0C, // PAYLOAD LENGTH = 12 bytes
0x55, // TYPE = 'U' (URI Record)
0x01, // URI Identifier Code (https://)
0x6C, 0x69, 0x6F, 0x6E, 0x6B, 0x65, 0x79, 0x2E, 0x64, 0x65, 0x76
/* lionkey.dev */
};
```

There are two handlers being used in the implementation. One for the `SELECT` command...

```
static uint16_t ndef_handle_select(t4t_context_t *ctx,
                                   const nfc_apdu_t *apdu,
                                   uint8_t *rsp)
{
    uint16_t fid;
    ...
    /* Select by file ID */
    if ((apdu->p1 == 0x00U) && (apdu->p2 == 0x0CU))
    {
        if ((ctx->selected_app < APP_NDEF) || (apdu->lc != 2U))
        {
            return nfc_put_sw(rsp, NFC_SW_FILE_NOT_FOUND);
```

```
        }

        fid = ((uint16_t)apdu->data[0] << 8) | apdu->data[1];

        if (fid == ctx->fid_cc)
        {
            ctx->selected_file = FILE_CC;
            return nfc_put_sw(rsp, NFC_SW_OK);
        }

        if (fid == ctx->fid_ndef)
        {
            ctx->selected_file = FILE_NDEF;
            return nfc_put_sw(rsp, NFC_SW_OK);
        }

        ctx->selected_file = FILE_NONE;
        return nfc_put_sw(rsp, NFC_SW_FILE_NOT_FOUND);
    }
    return nfc_put_sw(rsp, NFC_SW_FUNC_NOT_SUPPORTED);
}
```

...and the other for the `READ` command:

```
static uint16_t ndef_handle_read(const t4t_context_t *ctx,
                                 const nfc_apdu_t *apdu,
                                 uint8_t *rsp,
                                 uint16_t rsp_len)
{
    const uint8_t *src = NULL;
    uint16_t src_len = 0U;
    uint16_t offset = ((uint16_t)apdu->p1 << 8) | apdu->p2;
    uint16_t to_read = apdu->le;
    ...
    switch (ctx->selected_file) {
    case FILE_CC:
        src     = ctx->cc_file;
        src_len = ctx->cc_file_len;
        break;
    case FILE_NDEF:
        src     = ctx->ndef_file;
        src_len = ctx->ndef_file_len;
        break;
    default:
        return nfc_put_sw(rsp, NFC_SW_COND_NOT_SATISFIED);
    }

    if (offset > src_len)
    {
        return nfc_put_sw(rsp, NFC_SW_WRONG_PARAMS);
    }
```

```
    if ((uint32_t)offset + to_read > src_len)
    {
        to_read = (uint16_t)(src_len - offset);
    }

    if (rsp_len < (uint16_t)(to_read + 2U))
    {
        return nfc_put_sw(rsp, NFC_SW_NOT_ENOUGH_MEMORY);
    }

    return nfc_build_response(&src[offset],
                              to_read,
                              NFC_SW_OK,
                              rsp,
                              rsp_len);
}
```

Chapter 5
Results and Evaluation

5.1 Methodology

The evaluation process was structured in two stages, corresponding to different layers of the NFC communication stack.

First, the ability of the device to respond correctly during the activation phase was verified. This included transmission of fundamental identification and protocol parameters, namely ATQA (SENS_RES), UID, SAK, and ATS. The tests were performed using an external NFC reader under controlled conditions, enabling detailed observation of the anti-collision and activation sequence. Successful completion of this stage confirms correct operation at the physical and protocol initialization layers as defined by ISO/IEC 14443 and the ISO-DEP protocol.

In the second stage, the communication capabilities of the device were evaluated. Both supported application layers, CTAP and NDEF URI handling, were tested.

The mobile devices used for the testing were multiple iPhone devices ranging from iPhone 11 to iPhone 15 with iOS 18 to iOS 26, and a Samsung Galaxy S22+, representing the dominant mobile platforms, iOS and Android. Regarding personal computers, Apple does not allow the use of NFC security keys on macOS[1]. Consequently, a computer was used only for low-level functional testing through an external NFC reader, and not for real-world passkey authentication workflows.

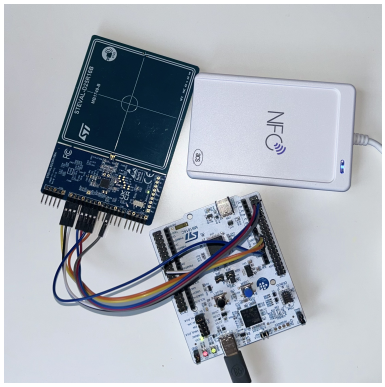

Figure 5.1. Evaluation Setup used for testing with computers.

[1] https://support.apple.com/en-us/102637

Test	Platform	Result	Notes
NFC-A activation	ACR122U + libnfc	Pass	–
FIDO conformance tests	ACR1552U + FIDO Tools	Pass	Complete test suite
CTAP2 over NFC	iPhone 11–15 (iOS 18–26)	Pass	Full CTAP2 interoperability
CTAP2 over NFC	Samsung Galaxy S22+	Fail	Falls back to U2F
NDEF URI read	iPhone 11–15 (iOS 18–26)	Pass	–
NDEF URI read	Samsung Galaxy S22+	Pass	–
NDEF URI read	ACR122U + NFC Tools	Pass	Manual APDU verification
WebAuthn registration	iPhone + Browser	Pass	–
WebAuthn authentication	iPhone + Browser	Pass	–

Table 5.1. Overview of evaluation tests, platforms, and results.

5.2 Response Testing

The basic communication parameters of the emulated card were verified using the `nfc-poll` utility from the `libnfc` library. The tool was used with an external NFC reader (ACS ACR122U) to observe the activation sequence.

The following values were obtained during the polling:

```
ATQA (SENS_RES): 00 44
UID (NFCID1):    5F 4C 4E 4B
SAK (SEL_RES):   20
ATS:             78 80 80 00
```

- The **ATQA** value (0x0044) indicates a Type A target using a single-size UID and participation in the anti-collision and selection procedure. The reported UID consists of four bytes: 5F 4C 4E 4B, which correspond to the ASCII sequence "_LNK". This static identifier was intentionally selected for testing and debugging purposes. In a production deployment, a unique identifier should be assigned to each hardware unit.
- The **SAK** (`0x20`) signals that the device implements the ISO-DEP protocol.
- The **ATS** sets the parameters of the upcoming transaction. The value of the lower nibble of the first byte, `0x78`, represents FSCI (FSC Integer–8–means the FSC is 256 bytes). `0x80`, as the value of the second byte, represents the bit rate; in this case, it is `ISODEP_ATS_TA_SAME_D`, meaning that the same bit rate is expected in both directions. The upper nibble of the third byte specifies 8 as WFI and the lower nibble 0 as SFGI.

Successful retrieval of these parameters demonstrates that the device correctly participates in the NFC-A activation sequence (see Section 3.6.1).

```
nfc-poll uses libnfc 1.8.0
NFC reader: ACS / ACR122U PICC Interface opened
NFC device will poll during 36000 ms (20 pollings of 300 ms for 6 modulations)
ISO/IEC 14443A (106 kbps) target:
    ATQA (SENS_RES): 00  44
       UID (NFCID1): 5f  4c  4e  4b
      SAK (SEL_RES): 20
                ATS: 78  80  80  00
Waiting for card removing...
```

Figure 5.2. Screenshot of the result of the nfc-poll command

5.3 Communication Capabilities Testing

5.3.1 CTAP

The correctness of the CTAP implementation was first verified using an external NFC reader through the FIDO Alliance Conformance Testing Tools application, which is available from the FIDO Alliance upon request. The tests include extended APDU CTAP requests, short APDU chaining (both in and out), and malformed APDU handling. The implementation passed all NFC-related test cases successfully. Since the original implementation had already been validated as FIDO-compliant, no major interoperability issues were expected with the NFC-enabled variant. This assumption was confirmed by executing the complete test suite previously used for the original implementation, with NFC used as the transport layer.

```
CE: start TX successful
Received APDU (10 bytes):8010000000000001040000
NFC: APDU parsed successfully
Handling extended APDU request: CLA=0x80 INS=0x10 P1=0x00 P2=0x00 Lc=1 Le=0
NFC: CTAP cmd=0x04 cbor_len=0
ctap_request: cmd=0x04 params_size=0
CTAP_CMD_GET_INFO
ctap_request: success in 6 ms, response length 176 bytes
```

Figure 5.3. Device log of processing an APDU containing a CTAP_GETINFO command.

We also tested the functionality of the implementation on the aforementioned mobile platforms.

For the iPhone devices, no deviations were observed from the expected behavior of the CTAP2 protocol, and the implementation exhibited complete interoperability with standard client platforms.

The tests conducted on the evaluated Samsung Galaxy S22+ did not demonstrate successful CTAP2 with respect to NFC interoperability. This may reflect the limitations of the platform, the vendor, or the software-stack rather than the universal Android ecosystem. The experimental evaluation indicates that the interaction is limited to the legacy U2F protocol (Universal Second Factor). While CTAP2 functionality was successfully verified when the LionKey device was connected via USB, attempts to establish CTAP2-based communication over NFC consistently failed. Instead, the Android system, upon receiving a response associated with the U2F protocol, issued commands consistent with this protocol, suggesting a fallback to legacy behavior in accordance with earlier FIDO Alliance specifications. Since LionKey only supports the CTAP2 standard, it is impossible to comply with Android devices at this point.

These observations suggest that, as of April 2026, Android devices do not provide practical support for CTAP2 over NFC for external authenticators, effectively limiting NFC-based communication to U2F compatibility. Although Google introduced the full FIDO2 API for Android in 2025[2], this support primarily targets platform authenticators, and external authenticators connected via USB and NFC support remains vendor-specific. Since the LionKey implementation targets FIDO2 exclusively and does not implement backward compatibility with U2F, this platform constraint prevents successful interoperability with Android devices in the NFC use case.

5.3.2 CTAP Latency and Reliability

The response time was measured for the most computationally demanding CTAP command, `CTAP_CMD_MAKE_CREDENTIAL`. Across 50 consecutive trials, the average response time was 42 ms in the production mode and 240 ms in the debug mode, which is comfortably within the maximum reply time suggested by the FIDO Alliance of 800 ms. The significant increase in debug mode can be attributed primarily to logging overhead and reduced optimization levels, which introduce additional processing delays.

Regarding reliability, it was observed that the initiating device does not consistently detect premature termination of an NFC session. Specifically, when the authenticator is removed from the field of the initiator during an ongoing authentication procedure, the device correctly transitions to an idle state. However, the client application continues to wait for a response and does not recover from interrupted communication.

As a consequence, subsequent authentication attempts cannot always be initiated, even though the user interface continues to indicate that NFC authentication is available. In this state, the NFC reader fails to re-establish communication with the authenticator, and recovery requires a full reset of the client context, typically by reloading the web page.

It was found that the frequency of this issue depends on the browser used. When tested with Safari, the failure occurred consistently after premature session termination. In contrast, Chrome and Firefox exhibited more robust behavior, requiring a page reload in approximately 40% of cases.

These observations suggest limited robustness in session termination and error handling within the client-side NFC stack. However, it cannot be conclusively determined to what extent this behavior is caused by the browser, the underlying operating system, or the implementation of the authenticator.

5.3.3 NDEF and APDU handling

The NDEF functionality was evaluated using a URI record to verify correct Type 4 Tag operation and interoperability with common NFC-enabled devices. The implementation was tested across multiple platforms, including iOS and Android devices, as well as an external NFC reader. NDEF communication flow is further explained in A.1.

We also verified the correct state machine behavior by manually sending APDUs that

[2] `https://developers.google.com/identity/fido/android/native-apps`

contained valid and invalid NDEF commands. This included checking for proper handling of file selection and data access as well as handling of invalid APDUs (i.e. APDUs not strictly following the structure referenced in 3.7.1.3.

5.3.4 NDEF Latency and Reliability

In all the scenarios tested, the NDEF message was successfully detected and processed, resulting in a 100% success rate in all 50 of the tests conducted. The communication followed the standard Type 4 Tag access sequence defined by the NFC Forum, consisting of application selection, capability container access, and NDEF file retrieval.

Response latency was measured at the APDU level in debug mode in 50 trials. The average response time was 25 ms for SELECT commands and 34 ms for READ BINARY commands. The increased latency for READ operations reflects the larger data payload and processing overhead associated with file retrieval. From the user perspective, no notable delay was observed during the interaction.

5.4 Use of CTAP on Real WebAuthn-enabled Websites

The implementation was further evaluated in real-world scenarios by testing it on WebAuthn-enabled websites. We managed to perform successful authentication on multiple websites, including the WebAuthn demo (see 5.4), Google, Amazon, or GitHub.

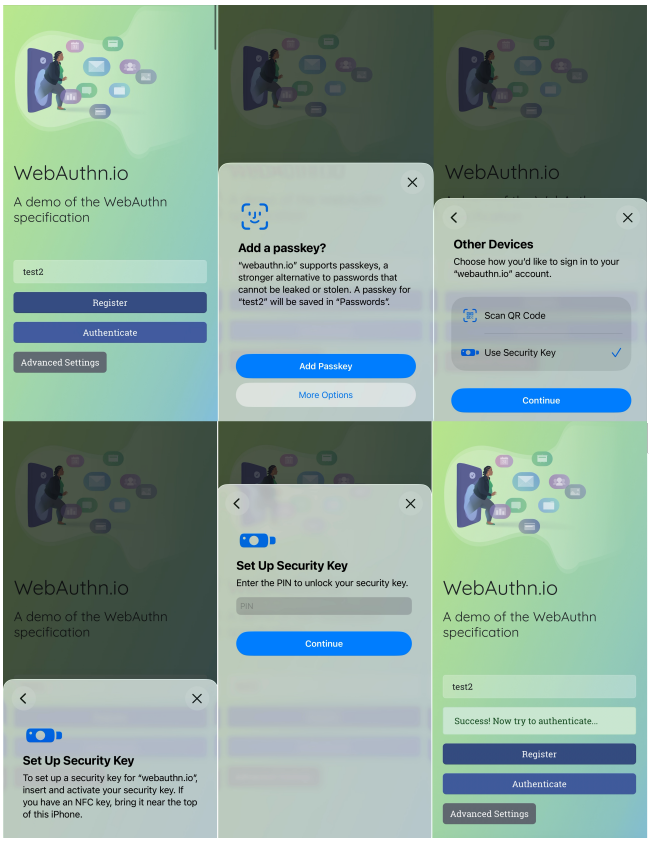

Figure 5.4. WebAuthn LionKey passkey registration flow via NFC on the iPhone on the WebAuthn Demo.

Chapter 6
Conclusion

This thesis aimed to extend the LionKey hardware authenticator with Near Field Communication (NFC) capabilities, enabling its use in contactless authentication scenarios based on the FIDO2 CTAP standard. To achieve this, an implementation of NFC card emulation using ISO-DEP was designed and integrated into the existing system architecture, as described in Chapter 4. The source code of the implementation is publicly available on GitHub[1].

The implemented solution was validated experimentally. As demonstrated in Chapter 5, the implementation supports both CTAP-based communication over NFC and NDEF-based data exchange. The correct functionality was confirmed at the protocol level by testing the APDU processing capabilities and at the application level through interoperability with client platforms.

These results come with certain practical constraints that are worth noting. In particular, the reliability of NFC-based authentication is affected by inconsistent authentication session termination handling on the iPhone, which may require manual recovery. The behavior differs from browser to browser. Furthermore, it was discovered that not all Android devices support CTAP2 over NFC. In addition, the current hardware platform does not support energy harvesting in combination with the required communication capabilities, limiting the feasibility of a fully passive device and ruling out commercial viability in its current form.

Addressing these constraints fully would require a secure element-based platform that would integrate both NFC and cryptographic functionality. Such a transition would necessitate a complete redesign of the current system architecture and is therefore beyond the scope of this thesis. This redesign can be set as a future goal for the development of the project along with transitioning to CTAP2.3, released in May 2026 [24], and adapting the key's algorithms to make the key quantum resistant.

In conclusion, the objectives of this thesis were met successfully. The presented implementation demonstrates that the integration of NFC-based communication into an existing hardware authenticator is feasible and provides a functional foundation for the further development of contactless security keys.

[1] `https://github.com/Matesin/lionkey-nfc`

References

[1] ITU. *Global and regional ICT data*. March, 2026.
https://www.itu.int/en/ITU-D/Statistics/Documents/facts/ITU_regional_global_Key_ICT_indicator_aggregates_Nov_2025.xlsx.

[2] Verizon. *Data Breach Investigations Report*. 2025.
https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf.

[3] Jumpcloud. *SME IT Trends Report*. 2024.
https://jumpcloud.com/wp-content/uploads/2024/02/202401-EB-SMEITTrendsReport.pdf.

[4] Sapio Research. *Consumer Password & Passkey Trends*. 2024.
https://fidoalliance.org/wp-content/uploads/2024/05/World-Password-Day-2024-Report-FIDO-Alliance.pdf.

[5] 6sense . *Market Share of Yubico*. 2025.
https://6sense.com/tech/two-factor-authentication/yubico-market-share.
Archived at:
https://perma.cc/MZ78-FJ62.

[6] Martin ENDLER. *FIDO2 USB bezpečnostní klíč*. 2025.

[7] Marco Casagrande, and Daniele Antonioli. *CTRAPS: CTAP Client Impersonation and API Confusion on FIDO2*. 2025.
https://hal.science/hal-05406456.

[8] Wikipedia contributors . *Post-quantum cryptography — Wikipedia, The Free Encyclopedia*. 2026.
https://en.wikipedia.org/w/index.php?title=Post-quantum_cryptography&oldid=1349019275. [Online; accessed 22-April-2026].

[9] Nina Bindel, Cas Cremers, and Mang Zhao. *FIDO2, CTAP 2.1, and WebAuthn 2: Provable Security and Post-Quantum Instantiation*. Cryptology ePrint Archive, Paper 2022/1029. 2022.
https://eprint.iacr.org/2022/1029.

[10] Quentin M. Kniep. *Post-Quantum FIDO2 Security Keys using Hash-Based Signatures*. 2021.
https://sar.informatik.hu-berlin.de/research/publications/SAR-PR-2021-05/SAR-PR-2021-05_.pdf.

[11] Mouser Electronics. *RFID & NFC, An Introduction to Near Field Communications*. 2026.
https://eu.mouser.com/applications/rfid-nfc-introduction/?srsltid=AfmBOorUoW7dRVaiDpD7tnmaVleDhgMmboEuJqqAB8Rxx17e4JZ_PlCT.

[12] ISO/IEC. *ISO/IEC 14443-2:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 2: Radio frequency power and signal interface*. 2020.

[13] ISO/IEC. *ISO/IEC 14443-1:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 1: Physical characteristics*. 2018.

[14] Yubico. *YubiKey Technical Manual*. 2026. `https://docs.yubico.com/hardware/yubikey/yk-tech-manual/webdocs.pdf`.

[15] Google LLC. *Titan Security Key*. 2026. `https://cloud.google.com/security/products/titan-security-key`.

[16] ISO/IEC. *ISO/IEC 18092:2023 Telecommunications and information exchange between systems — Near Field Communication Interface and Protocol 1 (NFCIP-1)*. 2023.

[17] Erik Hubers. *NFC Protocol Stack*. January 15, 2015. `https://commons.wikimedia.org/wiki/File:NFC_Protocol_Stack.png`.

[18] ISO/IEC. *ISO/IEC 14443-3:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 3: Initialization and anticollision*. 2018.

[19] ISO/IEC. *ISO/IEC 14443-4:2018 Cards and security devices for personal identification — Contactless proximity objects — Part 4: Transmission protocol*. 2018.

[20] ISO/IEC. *ISO/IEC 7816-4:2020 Identification cards — Integrated circuit cards — Part 4: Organization, security and commands for interchange*. 2020.

[21] FIDO Alliance. *Client to Authenticator Protocol (CTAP) Proposed Standard*. March 09, 2021. `https://fidoalliance.org/specs/fido-v2.1-rd-20210309/fido-client-to-authenticator-protocol-v2.1-rd-20210309.html`.

[22] STMicroelectronics. *STEVAL-25R3916B Data brief*. 2022. `https://www.st.com/resource/en/data_brief/steval-25r3916b.pdf`.

[23] ST Microelectronics. *RF/NFC abstraction layer (RFAL) User Manual*. February 10, 2026 . `https://www.st.com/resource/en/user_manual/um2890-rfnfc-abstraction-layer-rfal-stmicroelectronics.pdf`.

[24] FIDO Alliance. *Client to Authenticator Protocol (CTAP) Proposed Standard*. February 26, 2026. `https://fidoalliance.org/specs/fido-v2.3-ps-20260226/fido-client-to-authenticator-protocol-v2.3-ps-20260226.html`.

Appendix A
Referenced Figures

A.1 NDEF Communication Flow

The observed NDEF APDU flow consisted of the following steps:

1. SELECT command for the NDEF application identifier
2. SELECT command for the Capability Container (CC) file
3. READ BINARY command to retrieve the CC file
4. SELECT command for the NDEF file
5. READ BINARY command to retrieve the NDEF message.

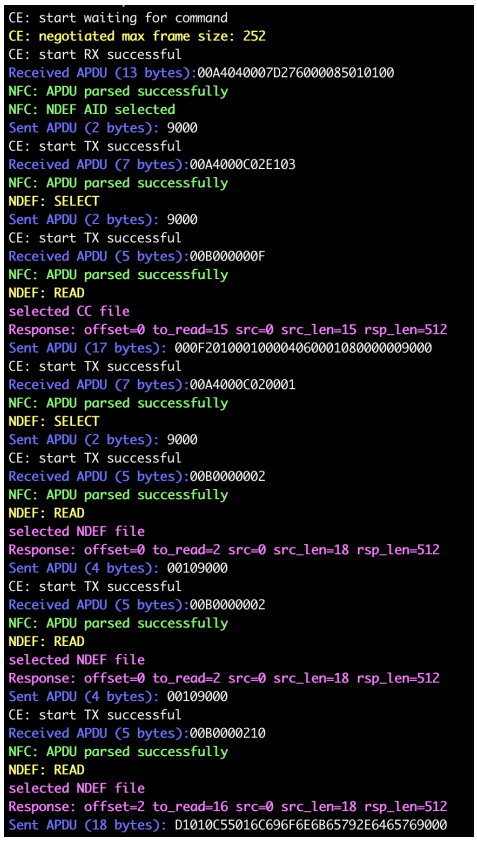

Figure A.1. Device Log of the NDEF Communication Flow.

Appendix B
Glossary

APDU	Application Protocol Data Unit
ASK	Amplitude Shift Keying
ATQA	Answer To reQuest (nfc-A)
ATS	Answer To Select
BPSK	Binary Phase Shift Keying
CE	Card Emulation
CID	Card Identifier
CL	Cascade Level
CLA	Class Byte
CTAP	Client to Authenticator Protocol
CTU FEE	Czech Technical University in Prague, Faculty of Electrical Engineering
DEP	Data Exchange Protocol
DID	Device Identifier
EM	Electromagnetic
fc	Carrier Frequency
FGT	Frame Guard Time
FIDO	Fast IDentity Online Alliance
FSC/FSD	Frame Size Card/Device
FWT	Frame Waiting Time
HB	Historical Bytes
HID	USB Human Interface Devices
IRQ	Interrupt Request
I2C	Inter-Integrated Circuit
LLCP	Logical Link Control Protocol
MCU	Microcontroller Unit
MFA	Multi-factor Authentication
NDEF	NFC Data Exchange Format
NFC	Near Field Communication
NRL-Z	Non-Return-to-Zero Leve
PCB	Printed Circuit Board
PCD	Proximity Coupling Device
PICC	Proximity Integrated Circuit Card
PQ	Post-Quantum
PSK	Phase Shift Keying
PXD	Proximity Extended Device
RATS	Request Answer To Select
REQA	REQuest, Type A
REQB	REQuest, Type B
RF	Radio Frequency
RFAL	RF Abstraction Layer
RFID	Radio-Frequency Identification

RFU	Reserved for Future Use
SE	Secure Element
SPI	Serial Peripheral Interface
SW	Status Word
UID	Unique IDentifier
USB	Universal Serial Bus
UV	User Verification
U2F	Universal 2nd factor
VHBR	Very High Bit Rate
WTX	Waiting Time Extension
2FA	Second-factor Authentication